

Object-Relational Mapping

Parte 4 - Transações



A

ATOMICITY

C

CONSISTENCY

I

ISOLATION

D

DURABILITY

Níveis de Isolamento

Serializable

Repeatable read

Read committed

Read uncommitted

Níveis de Isolamento

Serializable

Repeatable read

Read committed

Read uncommitted

- **Serializable:** Nível mais forte. É equivalente a uma ordem sequencial das transacções, não tendo anomalias. Obtém locks de todos os dados lidos e escritos até ao final da transacção, incluindo *SELECTs* com *WHEREs*.
- **Repeatable reads:** Semelhante, mas sem o lock nos *SELECTs* com *WHEREs*, e portanto sujeito a **phantom reads**.
`SELECT * FROM customer WHERE age > 21`
feito duas vezes na mesma transacção pode dar mais linhas da segunda vez.

Níveis de Isolamento

Serializable

Repeatable read

Read committed

Read uncommitted

- **Read committed:** Apenas mantém locks para escritas, libertando os locks dos reads. Ou seja, um SELECT pode dar resultados radicalmente diferentes na mesma transação (**non-repeatable reads**).
- **Read uncommitted:** Nível mais baixo. Uma transação pode ver dados que ainda não foram committed por outra transação (**dirty reads**).

Anomalias em Níveis de Isolamento

Read phenomenon Isolation level	Dirty read	Non-repeatable read	Phantom read
Serializable	no	no	no
Repeatable read	no	no	yes
Read committed	no	yes	yes
Read uncommitted	yes	yes	yes

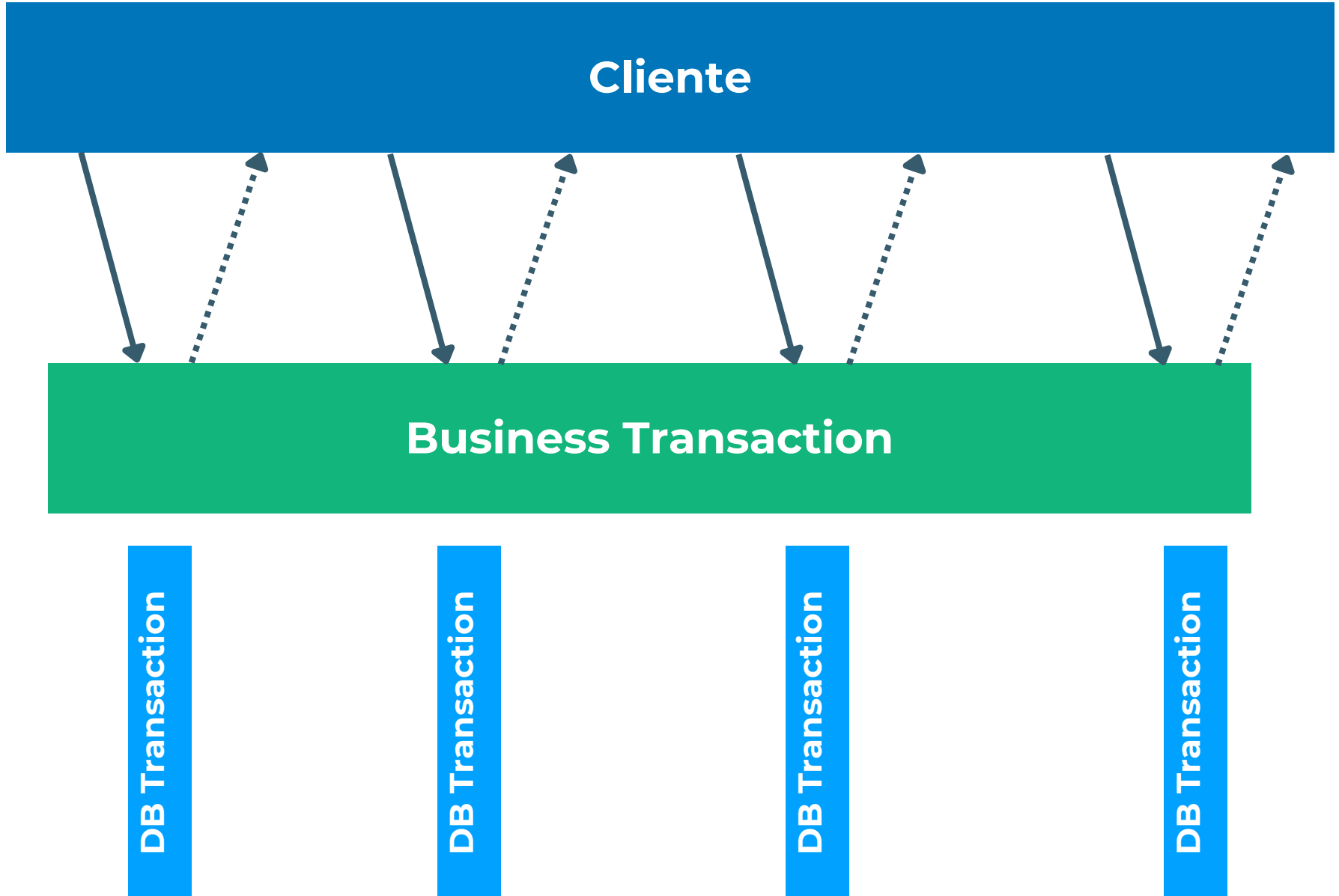
[https://en.wikipedia.org/wiki/Isolation_\(database_systems\)](https://en.wikipedia.org/wiki/Isolation_(database_systems))

Qual o problema com
os níveis de
isolamento altos?

Performance

- Transações precisam de locks.
- Locks tornam as aplicações lentas.
 - Sobretudo se esses dados forem *hot*
- Queremos transações curtas ao nível da BD.
- Mas as transações de negócio podem ser longas (Exemplo: marcar uma viagem de avião!)

Transacções de Negócio



Transacções de Negócio - Lost Update

Creditar 20€ à conta bancária

Cliente :
José

Saldo: 80€

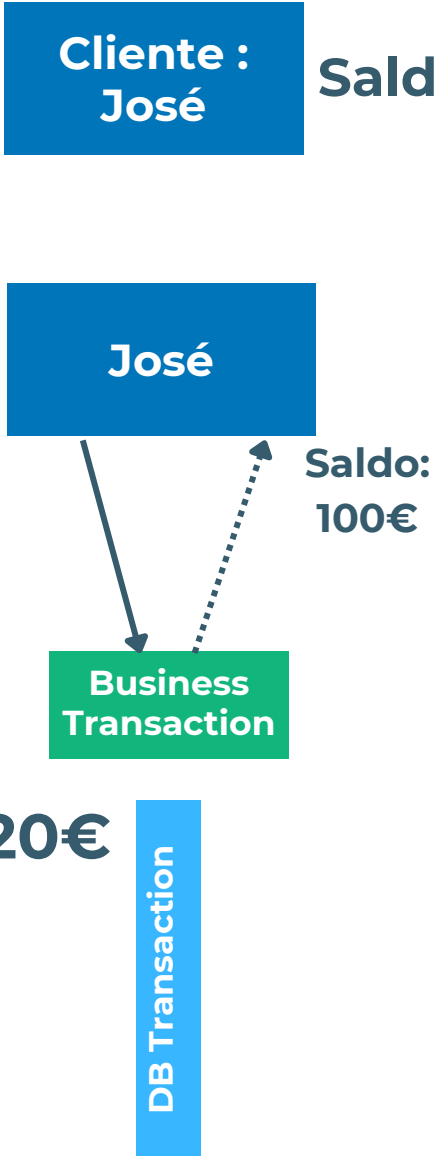
José

Saldo:
100€

Business
Transaction

+20€

DB Transaction



Transacções de Negócio - Lost Update

Creditar 20€ à conta bancária

Cliente :
José

Saldo: 80€



José

Business
Transaction

+20€

DB Transaction

José

Business
Transaction

+20€

DB Transaction

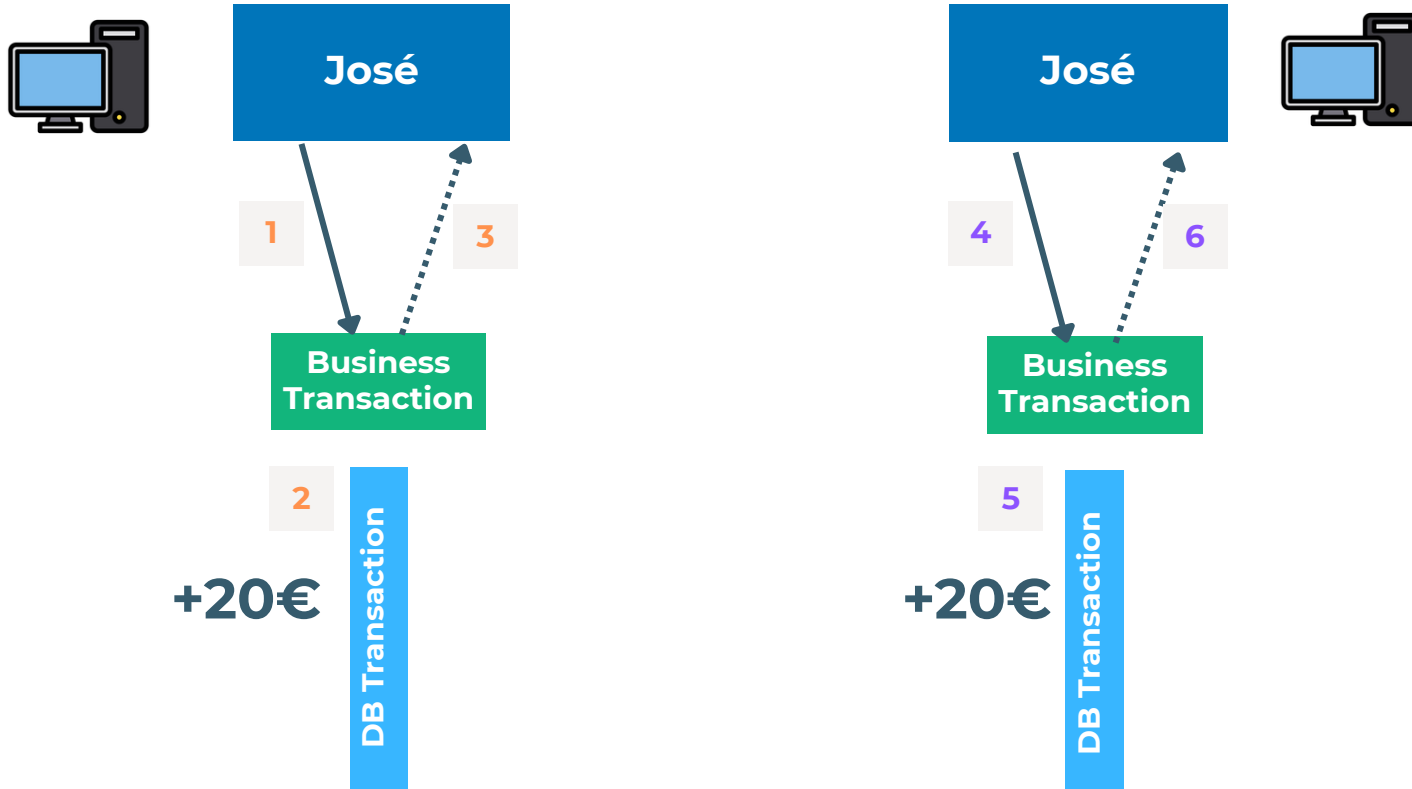


Transacções de Negócio - Lost Update

Creditar 20€ à conta bancária

Cliente :
José

Saldo: 80€



Transacções de Negócio - Lost Update

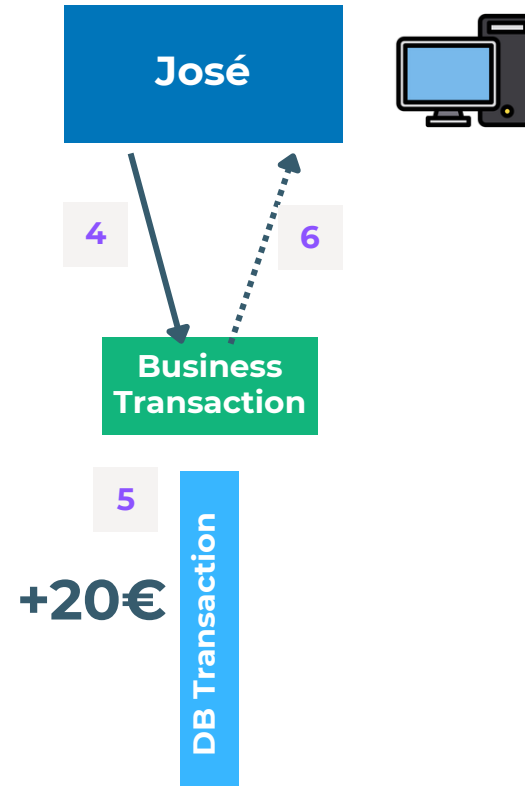
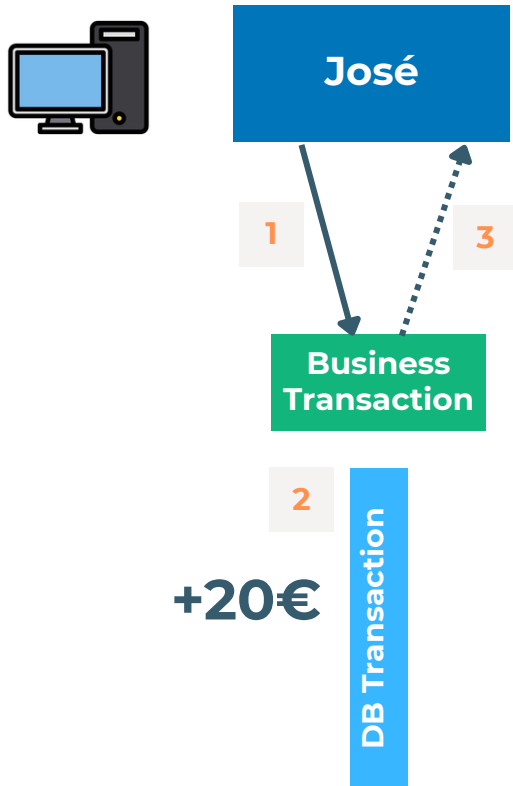
Creditar 20€ à conta bancária

Cliente :
José

Saldo: 80€

1 → 2 → 3 → 4 → 5 → 6

Saldo: 120€



Transacções de Negócio - Lost Update

Creditar 20€ à conta bancária

Saldo: 120€

1 → 2 → 3 → 4 → 5 → 6

Cliente :
José

Saldo: 80€

1 → 4 → 2 → 5 → 3 → 6

Saldo: 100€ □



José

1

3

Business
Transaction

2

+20€

DB Transaction

José

4

6

Business
Transaction

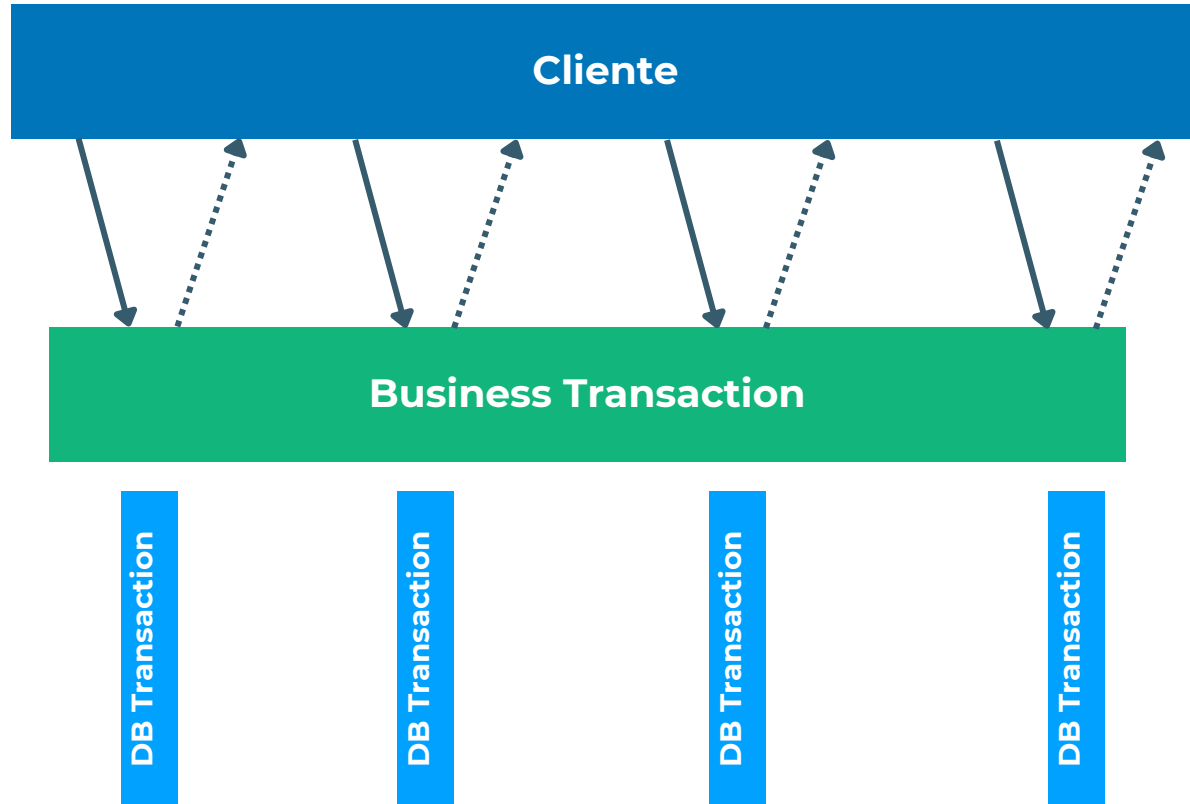
5

+20€

DB Transaction



Transacções de Negócio - Lost Update



Transacções de Negócio - Inconsistent Read

Salário do João e da Equipa

User: João
salary: 800

Equipa A

João

Ana

José



Cliente 1

getSalary
(João)

getSalary
(EquipaA)

Business Transaction

DB Transaction

DB Transaction

Cliente 2

updateSalary
(João, 1000)

Business Transaction

DB Transaction



Transacções de Negócio - Inconsistent Read

Salário do João e da Equipa

User: João
salary: 800

Equipa A

João

Ana

José

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9

1 → 7 → 8 → 9 → 2 → 3 → 4 → 5 → 6



Cliente 1

getSalary
(João)

1

Business Transaction

2

DB Transaction

getSalary
(EquipaA)

4

5

DB Transaction

3

6

Cliente 2

updateSalary
(João, 1000)

7

Business Transaction

8

DB Transaction



9

Transacções de Negócio - Inconsistent Read

Salário do João e da Equipa

User: João
salary: 800

Equipa A

João

Ana

José

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9

1 → 7 → 8 → 9 → 2 → 3 → 4 → 5 → 6



Cliente 1

getSalary
(João)

1

Business Transaction

2

DB Transaction

getSalary
(EquipaA)

4

5

DB Transaction

3

6

Cliente 2

updateSalary
(João, 1000)

7

Business Transaction

8

DB Transaction



9

Transacções de Negócio - Inconsistent Read

Salário do João e da Equipa

User: João
salary: 800

Equipa A

João

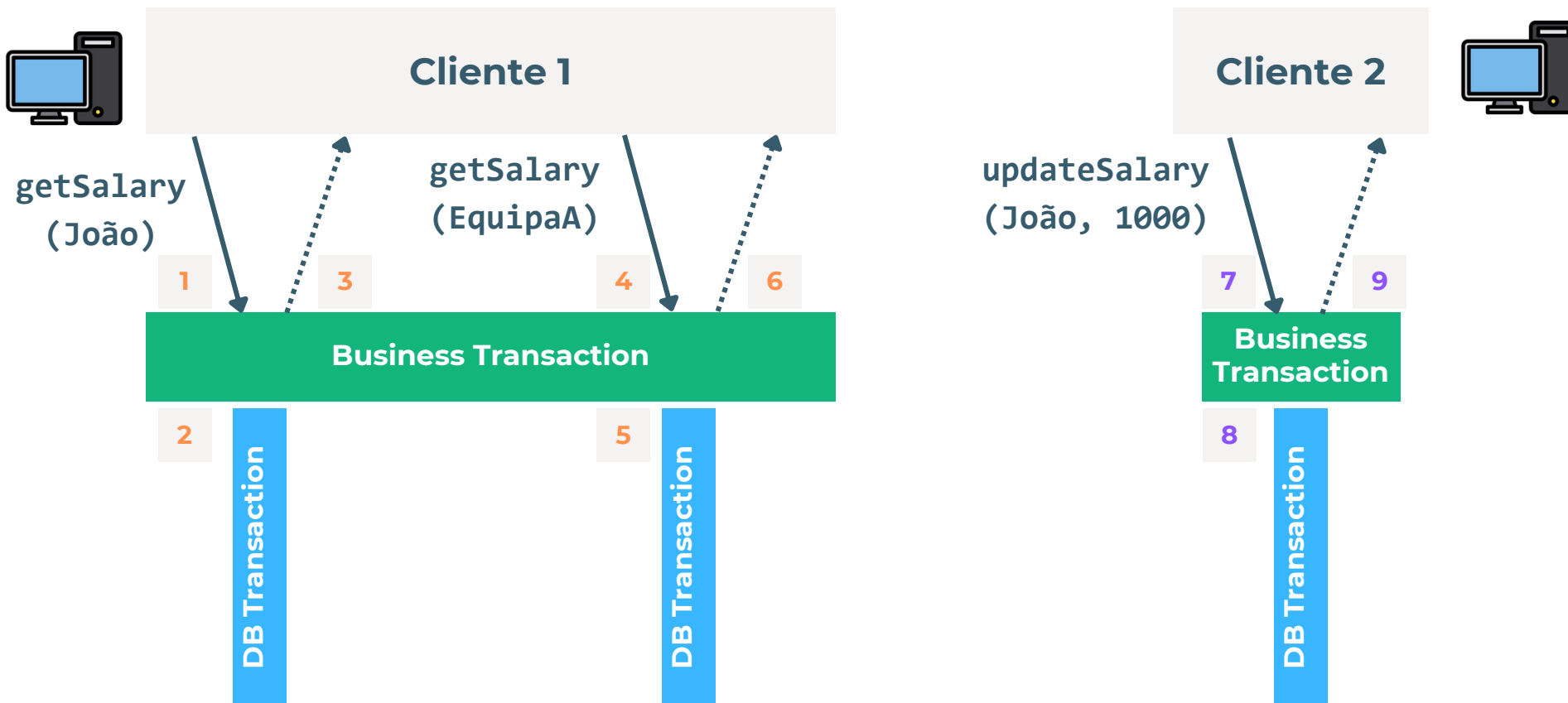
Ana

José

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9

1 → 7 → 8 → 9 → 2 → 3 → 4 → 5 → 6

1 → 2 → 3 → 7 → 8 → 9 → 4 → 5 → 6



Problemas comuns

- **Lost Update:** Quando uma transação escreve dados que foram calculados baseados em valores antigos (que foram entretanto substituídos por outra transação)
 - Exemplo: duas transações que dão um aumento de 20 euros à conta bancária, quando as escritas e leituras estão interleaved sem locks
- **Inconsistent Read:** Acontece quando se lêem dois valores que estão correctos por si só, mas não estão correctos em simultâneo
 - Exemplo: T1 lê o salário do João; T2 altera o salário do João; T1 lê o salário total da equipa do João. Ambas as respostas estão correctas isoladamente, mas não em simultâneo

Tipos de locks

- **Optimista**

- Assume que conflitos são raros, deixando as transações tentarem correr em paralelo
- Detecta conflitos (usando *versioning*) e aborta a transacção se for o caso

- **Pessimista**

- Assume que conflitos são comuns
- Transacções usam locks em dados até ao commit
- Podem causar deadlocks

Controlo de Concorrência no JPA

Version-based Concurrency Control

- Cada entidade tem um atributo anotado com `@Version`, que pode ser numérico ou timestamp
- A actualização e verificação das versões é gerada automaticamente pelo runtime do JPA
- Quando é feito um commit e não está na BD a versão esperada, é lançado uma `javax.persistence.OptimisticLockException`
- Pode ser necessário complementar com outras estratégias de locking

Version-based Concurrency Control

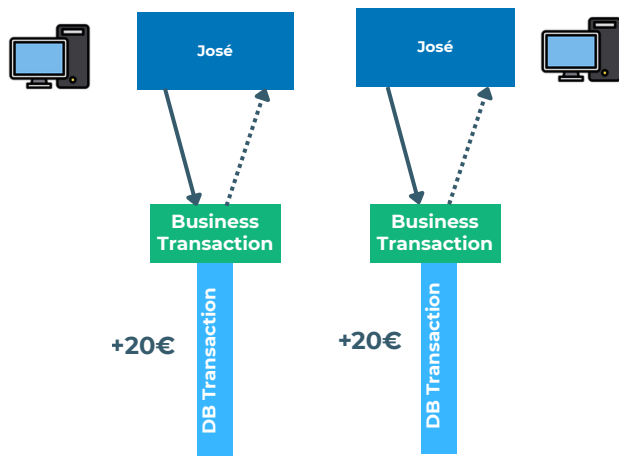
Update de versões

- Quando é mudado um atributo simples
- Quando é alterada uma associação *owned* pela entidade, incluindo as registadas em JOIN TABLES
- A operação `.merge()` garante que a versão do objecto que recebe está actualizada e não *stale*. Se estiver *stale*, levanta uma `OptimisticException`
- Comparar versões **apenas funciona dentro da mesma entidade**
 - Se usamos valores entre entidades, precisamos de locks explícitos!

Version-based Concurrency Control

Update de versões

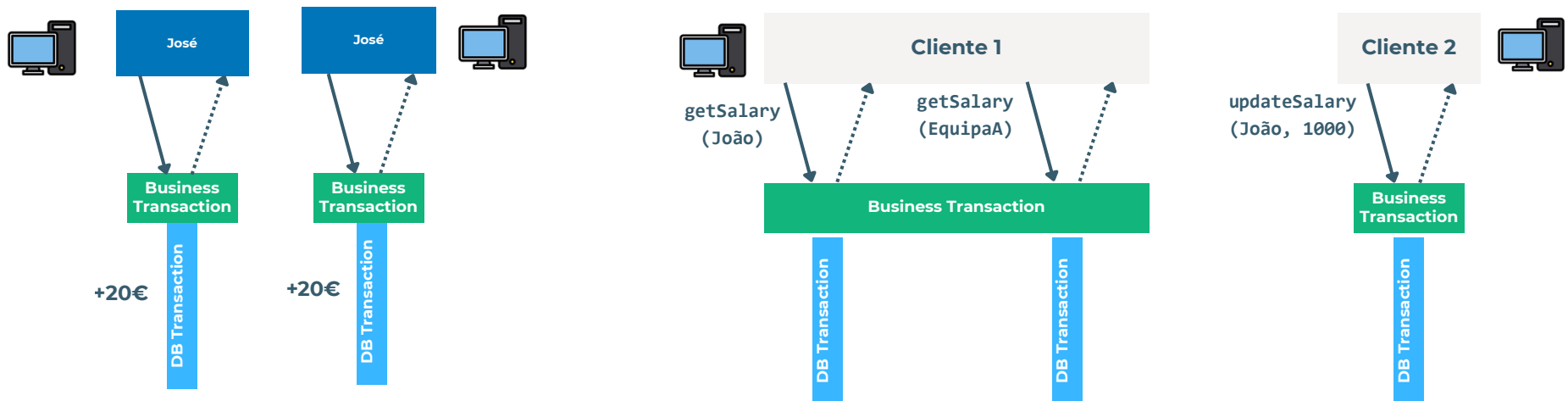
- Comparar versões **apenas funciona dentro da mesma entidade**
 - Se usamos valores entre entidades, precisamos de locks explícitos!



Version-based Concurrency Control

Update de versões

- Comparar versões **apenas funciona dentro da mesma entidade.**



Version-based Concurrency Control

Locking explícito

- `EntityManager.lock( obj, mode )`
- `EntityManager.find( ..., mode )`
- `EntityManager.refresh( ..., mode )`
- `query.setLockMode(mode)`

Version-based Concurrency Control

Modos de Locking

- **OPTIMISTIC**: O número de versão é verificado imediatamente antes do commit
- **OPTIMISTIC_FORCE_INCREMENT**: igual ao optimistico, mas garante que o número de versão aumenta sempre
- **PESSIMISTIC_READ**: Usa um lock de leitura na DB. Permite várias leituras concorrentes, mas nenhuma escrita
- **PESSIMISTIC_WRITE**: Usa um lock de escrita na DB. Mais nenhuma operação concorrente é permitida

@Transactional

- Define que uma classe ou método deve ser tratada como transação
 - Permite definir o tipo específico de isolamento a nível de base de dados
 - `@Transactional(isolation = Isolation.SERIALIZABLE)`
 - Por defeito, tem rollback para *RuntimeExceptions*
 - *Checked Exceptions* não causam rollback
 - Pode ser alterado com base em anotações

@Transactional

- Propagam o efeito de transação para métodos aninhados
 - Por defeito, executam na mesma transação
 - `Propagation.REQUIRED` → Se uma transação não existir, cria-se uma nova
 - Alternativa: `Propagation.REQUIRES_NEW` → Cria sempre uma nova transação
 - Anotações em métodos mais internos tem prioridade sob anotações herdadas
 - `@Transactional(readOnly = true)`
 - `readOnly` faz com que o Hibernate compare o estado da entidade BD no final da transação → +performance

@Transactional

- A transação só começa se o método **inicial dentro da mesma classe** estiver anotado
 - `Clazz.foo()` → `Clazz.boo()`
 - Se boo estiver anotado mas foo não, a **transação não começa**
- Todos os métodos do `JpaRepository` são anotados com `@Transactional` por defeito
- É possível definir um tempo máximo de execução por transação para evitar transações com problemas ou muito longas

Recapitulando

- **Identity Map**: controla o carregamento de entidades, para que nunca existam dois objectos para a mesma linha
- **Unit of Work**: objecto que mantém a lista de instâncias de entidades que foram alteradas por uma transação, e lida com a gestão de concorrência
- **Lazy Load**: atributos de objectos que são trazidos da BD apenas quando se acedem
- **Optimistic offline lock**: valida que os objectos que vão ser committed foram obtidos a partir das versões mais recentes na BD
- **Query object**: criação de queries SQL dinamicamente

Object-Relational Mapping

Parte 4 - Transações